

CSCI-4208: Developing Advanced Web Applications

Week 8: Lecture 12

Web Sockets

Overview: Web Sockets

- HTTP Limitations
- HTTP Workarounds
- WebSockets vs HTTP
- WebSockets - Browser-side
- WebSockets - Server-side
- WebSockets - Protocol
- Socket.io
- Socket.io - Analogy
- Socket.io - Methods Overview
- Socket.io - Demo

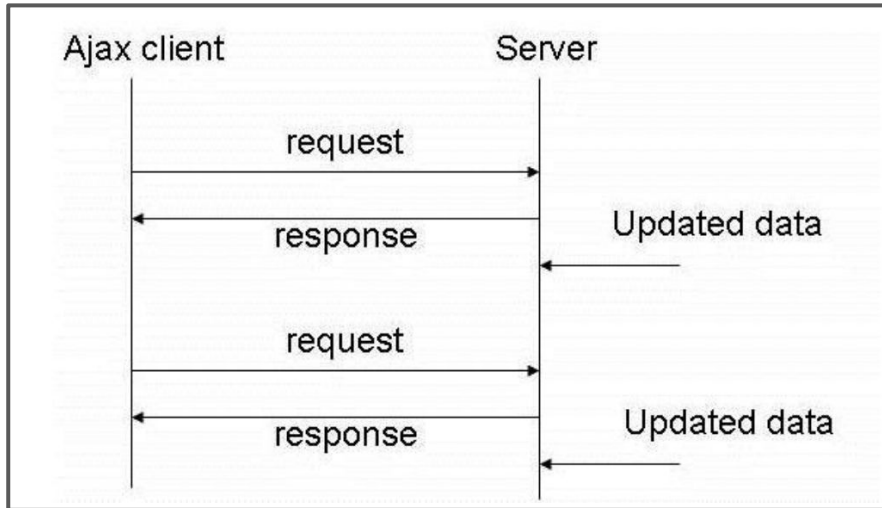
HTTP / AJAX - Limitations

- HTTP is request/response protocol
- AJAX allows you to do HTTP request/response within a page, but it is still request/response
- A client always has to initiate communication. – I.e. Make the request
- What if you want the server to be able to "push" information down to the client, for example:
 - New email
 - New news/notifications
 - Updates to a shared chat / drawing canvas
 - Game events

HTTP Work Arounds

HTTP Workarounds have been devised:

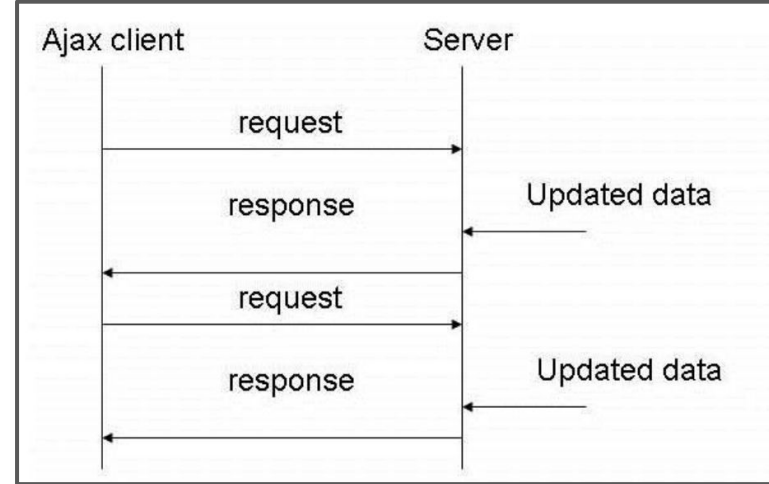
- E.g. Polling
 - Client continuously polls the server



HTTP Work Arounds

HTTP Workarounds have been devised:

- E.g. Long Polling
 - Client sends a request, which the server does not respond to until it has something to send
 - If something comes along to send, it completes the response and the client will immediately send a new request to hold on to for later
 - If the request timed out, the client sends a new response.



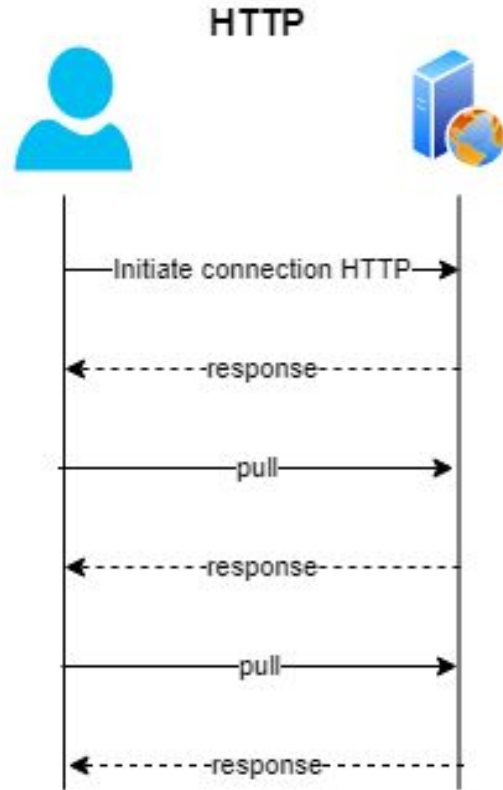
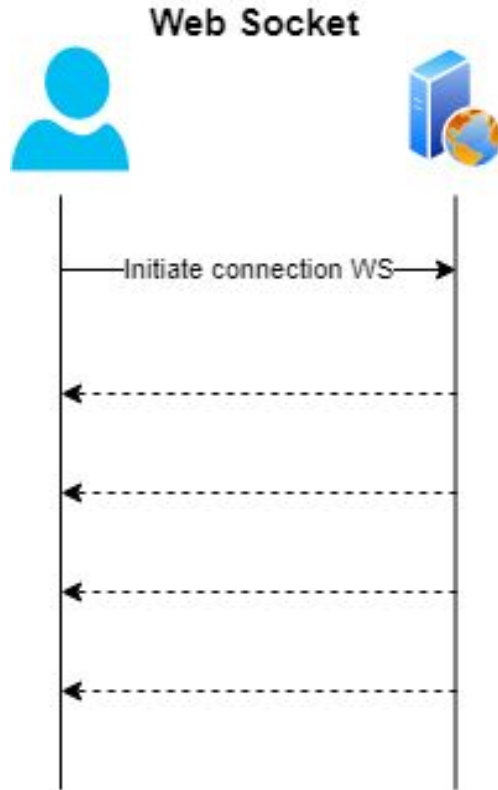
Analogy (from the days of postal mail)...

- Cheap friend can't afford stamps.
- Send friend stamp, he uses it to send you a letter
- When you receive message, immediately send him another stamp for the next time he wants to send a letter.

WebSockets

- Provides true **two-way ongoing** communication between a client & server.
- Each side can "**send**" messages.
- Each side has listener "**on message**"
- WebSockets are not restricted by the **Same Origin Policy**
 - It is up to the server to check the Origin header and decide whether it should reply or not.

WebSockets



WebSockets - On the browser

Browsers natively support WebSockets from the global window object:

Example:

```
// Create a new WebSocket
var wSocket = new WebSocket("ws://www.example.com/socketserver")

// When the WebSocket is connected and ready
wSocket.onopen = function (event) {
    // Send a message to the server
    wSocket.send("Initial message to the server.");
};

// Handler for when a message arrives from the server
wSocket.onmessage = function (event) {
    console.log( event.data );
};
```

WebSockets - Server-side

- Require a WebSockets module
 - ws is very popular
 - <https://www.npmjs.com/package/ws>
- Create a new WebSockets Server
- Define handlers for
 - onconnection
 - on message
- And *send* messages to the client

WebSockets - Protocol

- URL Prefix
 - ws:// for normal connections
 - wss:// for secure connections

WebSockets - Protocol

- HTTP Request for initializing WebSocket Connection

```
GET ws://echo.websocket.org/?encoding=text HTTP/1.1
Origin: http://websocket.org
Cookie: __utma=99as
Connection: Upgrade
Host: echo.websocket.org
Sec-WebSocket-Key: uRovscZjNol/umbTt5uKmw==
Upgrade: websocket
Sec-WebSocket-Version: 13
```

- HTTP Response for initializing WebSocket Connection

```
HTTP/1.1 101 WebSocket Protocol Handshake
Date: Fri, 10 Feb 2012 17:38:18 GMT
Connection: Upgrade
Server: Kaazing Gateway
Upgrade: WebSocket
Access-Control-Allow-Origin: http://websocket.org
Access-Control-Allow-Credentials: true
Sec-WebSocket-Accept: rLHCkw/SKs09GAH/ZSFhBATDKrU=
Access-Control-Allow-Headers: content-type
```

WebSockets - Protocol

- After handshake:
 - HTTP connection broken down
 - Replaced by WebSocket connection
- Over the same TCP/IP connection
 - Upgrade is one way, can't go back to HTTP

Socket.io

- Library for full-duplex communication between clients and servers
- Built on engine.io, socket.io provides an abstraction above whatever transport methods are available in the browser:
 - WebSockets
 - Long Polling

Socket.io

- Provides a simple abstraction for event-based programming **across the server and multiple clients**
- Clients can:
 - emit events back to the server
 - listen for events from the server
- Server can:
 - listen for events from clients
 - emit events back to the client
 - emit events to other clients
 - broadcast events to all clients

Socket.io (D&D Analogy)

- At each table, create a game
- An event is simply a word
 - It can optionally be accompanied with a short phrase
- One person is the Server who can:
 - listen for events from clients
 - emit an event to one client (point to one and say...)
 - broadcast emit events to all clients (point with both hands and say...)
- The rest at the table are Clients who can
 - emit events to the server
 - listen for events from the server
- Clients or the Server can generate events at any time (depending on your game-play)
- Clients cannot emit events peer-to-peer (Server must relay)

Socket.io - Methods overview

- `on(event, function(parameter-object) { ... })`
- `emit(event, parameter-object)`
- `events`
 - `connection`
 - `disconnect`
 - *developer-defined events*

Socket.io - Methods overview

- `socket.emit(...)`
 - emits back to the client
- `socket.broadcast.emit(...)`
 - emits to all connected clients except the socket that starts it

Socket.io - Demo

- Simple Demo - Chat App
 - Code walkthrough of Chat App
- Complex Demo - Monster Catcher game
 - Code walkthrough of Monster Catcher game

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

Chat App Lab

Homework

Create a WebSocket App